

LLM vs Agent — 知识图谱

《从零构建你的 AI Agent》Ch 1 · 从一次 API 调用开始

LLM（大语言模型）

stateless · single-call · language I/O only



局限性

- ❌ 无状态 - 每次调用独立, 彼此不认识
- ❌ 无工具 - 不能主动调 API、查数据库
- ❌ 无循环 - 只做一问一答, 不能多步规划
- ❌ 知识截止 - 无法获取训练后的新信息
- ✅ 极速 - 单次调用毫秒级, 成本低

Python 调用模式

```
response = client.messages.create(  
    model="claude-opus-4-5-20251101",  
    messages=[{"role": "user", "content": prompt}],  
)
```

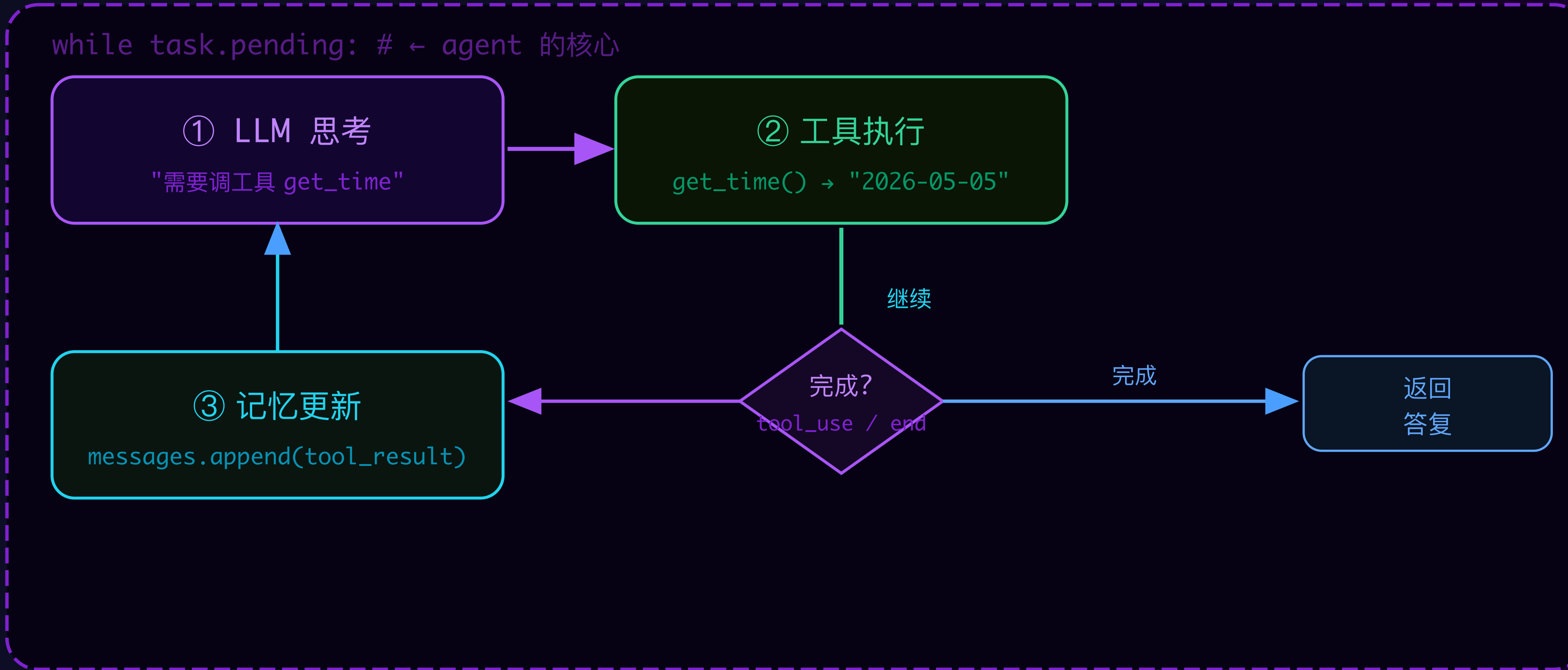
一次调用结束, 没有循环, 没有工具

升级路径

- +Loop
- +Tools
- +Mem

Agent（智能体）

loop + tools + memory · autonomous



能力

- ✅ 有循环 - 多步决策, 不止一次调用
- ✅ 有工具 - 查时间/搜网页/读文件/执行代码
- ✅ 有记忆 - 每步结果都加进上下文
- ✅ 有主动性 - 不等你问, 定时推送晨报

三家 API 请求格式对比矩阵

同样调一次"你好", 三家结构差异一目了然

Anthropic

```
# Anthropic Messages API  
client.messages.create(  
    model="claude-opus-4-5-20251101",  
    max_tokens=1024,  
    system="你是 Lena", ← 顶层字段  
    messages=[  
        {"role": "user",  
         "content": "你好"} ← 字符串  
    ]  
)  
  
# 回复路径:  
response.content[0].text  
  
stop_reason: "end_turn"  
usage.input_tokens / output_tokens
```

OpenAI

```
# OpenAI Chat Completions  
client.chat.completions.create(  
    model="gpt-4o",  
    messages=[  
        {"role": "system", ← messages 里  
         "content": "你是 Lena"},  
        {"role": "user",  
         "content": "你好"} ← 字符串  
    ]  
)  
  
# 回复路径:  
response.choices[0].message.content  
  
finish_reason: "stop"  
usage.prompt_tokens / completion_tokens
```

AWS Bedrock

```
# AWS Bedrock Converse API  
bedrock.converse(  
    modelId="us.anthropic.claude-sonnet-4-6",  
    # ← inference profile ID (非 model ID)  
    system=[{"text": "你是 Lena"}], ← 列表  
    messages=[  
        {"role": "user",  
         "content": [{"text": "你好"}]} ← 列表  
    ],  
    inferenceConfig={"maxTokens": 1024},  
)  
  
# 回复路径 (多层嵌套):  
resp["output"]["message"]["content"][0]["text"]  
  
stopReason: "end_turn" · usage.inputTokens
```